

Solucionario Técnico: Bloque 1

Ficheros de Texto Plano, Charsets, Buffers y API Moderna Java NIO.2

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Docente · Solución Oficial Comentada

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.a (Clases de E/S y Flujos)

CE1.b (Ficheros de Texto Plano)

Guía de resolución exhaustiva con código estándar Java 21 para los ejercicios de validación de ficheros CSV heterogéneos, generación de informes con `PrintWriter` formateado y análisis forense reactivo con Streams de Java NIO.2.

Claves Técnicas de la Implementación:

- **Lectura en ISO-8859-1** : Se abre con `Files.newBufferedReader(path, StandardCharsets.ISO_8859_1)` garantizando la interpretación fiel de tildes y eñes sin sustitución por caracteres anómalos.
- **Uso de `split(";", -1)`** : El parámetro `-1` preserva los campos vacíos al final de la línea para detectar columnas faltantes.
- **Canalización de descartes**: Se segregan los registros limpios en `UTF-8` y los descartes en `.log` sin interrumpir el flujo.

Código Fuente Completo:

```

package soluciones_b1;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class ValidadorCSV {

    public static void main(String[] args) {
        Path rutaOrigen = Path.of("datos", "empleados_raw.csv");
        Path rutaDestino = Path.of("datos", "empleados_limpios.csv");
        Path rutaErrores = Path.of("datos", "empleados_errores.log");

        int lineasValidas = 0;
        int lineasRechazadas = 0;
        int numeroFila = 0;

        try (BufferedReader br = Files.newBufferedReader(rutaOrigen, StandardCharsets.ISO_8859_1);
            BufferedWriter bwLimpios = Files.newBufferedWriter(rutaDestino, StandardCharsets.UTF_8);
            BufferedWriter bwErrores = Files.newBufferedWriter(rutaErrores, StandardCharsets.UTF_8)) {

            String linea;
            // Preservar y trasladar la cabecera
            if ((linea = br.readLine()) != null) {
                numeroFila++;
                bwLimpios.write(linea);
                bwLimpios.newLine();
            }

            while ((linea = br.readLine()) != null) {
                numeroFila++;
                if (linea.trim().isEmpty()) continue;

                String[] campos = linea.split(";", -1);
                if (campos.length != 5) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: Columnas insuficientes (%d/5) -",
                        numeroFila, campos.length, linea));
                    lineasRechazadas++;
                    continue;
                }

                String dni = campos[0].trim();
                String nombre = campos[1].trim();
                String departamento = campos[2].trim();
                String salarioStr = campos[3].trim();
                String fechaAlta = campos[4].trim();

                if (dni.isBlank()) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: DNI vacío o en blanco -> %s%n",
                        numeroFila, linea));
                    lineasRechazadas++;
                    continue;
                }

                double salario;
                try {
                    salario = Double.parseDouble(salarioStr);
                    if (salario <= 0) throw new NumberFormatException("Salario <= 0");
                } catch (NumberFormatException e) {
                    bwErrores.write(String.format("[LÍNEA %d] Error: Salario no válido (%s)\n",
                        numeroFila, salarioStr));
                    lineasRechazadas++;
                    continue;
                }

                bwLimpios.write(linea);
                bwLimpios.newLine();
                lineasValidas++;
            }
        }
    }
}

```

```

    } catch (NumberFormatException e) {
        bwErrores.write(String.format("[LÍNEA %d] Error: Salario inválido ('%s') -> %s%n",
            numeroFila, salarioStr, linea));
        lineasRechazadas++;
        continue;
    }

    // Escribir registro normalizado
    bwLimpios.write(String.format("%s;%s;%s;%s;%s;%s", dni, nombre, departamento, salario,
        direccion, telefono));
    bwLimpios.newLine();
    lineasValidas++;
}

System.out.printf("✅ Proceso completado: %d válidos, %d rechazados.%n", lineasValidas,
    lineasRechazadas);

} catch (IOException e) {
    System.err.println("Error crítico de E/S: " + e.getMessage());
}

}
}

```

Claves Técnicas de la Implementación:

- **Uso de `Locale.US` en `printf`** : Garantiza formato numérico con punto decimal homogéneo si se requiere o coma estándar sin alterar la alineación.
- **Alineación precisa:** `%-6s` para strings a la izquierda, `%10.2f €` y `%8d` para campos numéricos y monetarios a la derecha.
- **Backup automático en NIO.2:** Empleo de `Files.copy()` con `StandardCopyOption.REPLACE_EXISTING` una vez cerrado el flujo de escritura.

Código Fuente Completo:

```

package soluciones_b1;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.Locale;

public class GeneradorInforme {

    public static void main(String[] args) {
        Path entrada = Path.of("datos", "ventas_diarias.txt");
        File salida = new File("datos/informe_ventas.txt");
        Path backup = Path.of("datos", "backups", "informe_ventas_backup.txt");

        double totalFacturacion = 0;
        int totalUnidades = 0;
        int numTiendas = 0;
        String tiendaLider = "";
        double maxVentas = -1;

        try (BufferedReader br = Files.newBufferedReader(entrada, StandardCharsets.UTF_8);
             PrintWriter pw = new PrintWriter(new BufferedWriter(new FileWriter(salida, StandardCharsets.UTF_8)))) {

            pw.println("=====");
            pw.println("                        INFORME DIARIO DE VENTAS - CADENA ASTUR RETAIL");
            pw.println("=====");
            pw.printf(Locale.US, "%-6s %-20s %14s %10s %14s%n", "ID", "CIUDAD", "IMPORTE NETO", "UNIDADES", "TICKET MEDIO");
            pw.println("-----");

            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.isBlank() || linea.startsWith("#")) continue;

                String[] p = linea.split("\\|");
                String id = p[0].trim();
                String ciudad = p[1].trim();
                double importe = Double.parseDouble(p[2].trim());
                int unidades = Integer.parseInt(p[3].trim());

                double ticketMedio = unidades > 0 ? (importe / unidades) : 0;
                totalFacturacion += importe;
                totalUnidades += unidades;
                numTiendas++;

                if (importe > maxVentas) {
                    maxVentas = importe;
                    tiendaLider = ciudad;
                }

                pw.printf(Locale.US, "%-6s %-20s %12.2f € %10d %12.2f €%n",
                    id, ciudad, importe, unidades, ticketMedio);
            }

            double mediaPorTienda = numTiendas > 0 ? (totalFacturacion / numTiendas) : 0;

            pw.println("=====");
            pw.printf(Locale.US, "TOTALES: %d Tiendas | Facturación Total: %12.2f € | Unidades: %d | Media por tienda: %12.2f €%n",
                numTiendas, totalFacturacion, totalUnidades, mediaPorTienda);
        }
    }
}

```

```
        pw.printf(Locale.US, "PROMEDIO POR TIENDA: %12.2f € | TIENDA LÍDER: %s (%.2f €)%n",
            mediaPorTienda, tiendaLider, maxVentas);
        pw.println("=====");

        System.out.println("✅ Informe generado en: " + salida.getPath());

    } catch (IOException e) {
        System.err.println("Error procesando ventas: " + e.getMessage());
        return;
    }

    // Copia de seguridad atómica
    try {
        Files.createDirectories(backup.getParent());
        Files.copy(salida.toPath(), backup, StandardCopyOption.REPLACE_EXISTING);
        System.out.println("📦 Backup atómico generado en: " + backup);
    } catch (IOException e) {
        System.err.println("Error creando backup: " + e.getMessage());
    }
}
```

Solución Ejercicio 3: Analizador Forense con NIO.2 y Streams

(LogAnalyzerNIO.java)

NIVEL
AVANZADO

Claves Técnicas de la Implementación:

- **Consumo de memoria $O(1)$:** `Files.lines()` lee secuencialmente bajo demanda. No se instancia ninguna lista completa en la memoria heap.
- **Filtrado y volcado con `peek()`** : Permite escribir las líneas anómalas en el archivo de alertas al mismo tiempo que se cuentan.
- **Agrupación y ranking:** `Collectors.groupingBy(..., Collectors.counting())` y ordenación con `comparingByValue().reversed()` y `limit(3)`.



Código Fuente Completo:

```

package soluciones_b1;

import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class LogAnalyzerNIO {

    public static void main(String[] args) {
        Path logPath = Path.of("datos", "access.log");
        Path alertasPath = Path.of("datos", "peticiones_sospechosas.txt");

        try {
            // 1. Conteo total de peticiones
            try (Stream lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {
                System.out.println("1. Total peticiones auditadas: " + lineas.count());
            }

            // 2. Filtrado de errores HTTP >= 400 y volcado a alertas
            try (Stream lineas = Files.lines(logPath, StandardCharsets.UTF_8);
                PrintWriter pw = new PrintWriter(Files.newBufferedWriter(alertasPath, StandardCharsets.UTF_8))) {

                long errores = lineas.filter(l -> {
                    String[] partes = l.split(" ");
                    if (partes.length >= 9) {
                        try {
                            int status = Integer.parseInt(partes[partes.length - 2]);
                            return status >= 400;
                        } catch (NumberFormatException ignored) {}
                    }
                    return false;
                }).peek(pw::println).count();

                System.out.println("2. Peticiones de error (>=400) volcadas a alertas: " + errores);
            }

            // 3. Suma de bytes transmitidos en 200 OK
            try (Stream lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {
                long bytesTotales = lineas
                    .filter(l -> l.contains("\" 200 "))
                    .mapToLong(l -> {
                        String[] partes = l.split(" ");
                        try {
                            return Long.parseLong(partes[partes.length - 1]);
                        } catch (Exception e) {
                            return 0L;
                        }
                    })
                    .sum();

                System.out.printf("3. Bytes totales transmitidos con éxito: %,d bytes%n", bytesTotales);
            }

            // 4. Top 3 Direcciones IP más activas
            try (Stream lineas = Files.lines(logPath, StandardCharsets.UTF_8)) {

```

```

        System.out.println("4. Top 3 Direcciones IP más activas:");
        lineas.map(l -> l.split(" ")[0].trim())
            .collect(Collectors.groupingBy(ip -> ip, Collectors.counting()))
            .entrySet().stream()
            .sorted(Map.Entry.comparingByValue().reversed())
            .limit(3)
            .forEach(e -> System.out.printf("    - IP: %-15s -> %d peticiones%n", e.getKey(), e.getValue()));
    }

} catch (IOException e) {
    System.err.println("Error en lectura NIO.2: " + e.getMessage());
}

}

}

```